

NexLink

Demo Guide

Step-by-step script for presenting the complete NexLink project

Item	Detail
Presenter	Maitrey Bharambe
Prepared on	24 October 2026
Demo length	About 25 minutes + questions (10 acts; acts marked optional can be skipped)
Equipment	One Windows PC with this repository, 2–3 PowerShell windows, a browser. A second computer is optional (Act 6c).
Device names used	Real names: this PC (hostname Maitrey) enrolls as maitrey, a second agent on it as maitrey-peer; a second computer uses its own hostname
Readiness	Components checked on 24 Oct 2026 — see section 1

Contents

Demo at a glance ... 3
1. Readiness check (24 October 2026) ... 4
2. The day before ... 5
2.1 Update and install ... 5
2.2 Use a clean demo database ... 5
2.3 Decide on AI (Acts 7 and 8) ... 5
2.4 Rehearse once ... 5
3. Starting everything (10 minutes before) ... 6
Quick checks ... 6
4. The demo, act by act ... 7
Act 1 — Landing page and architecture (3 min) ... 7
Act 2 — Sign up and guided setup (3 min) ... 7
Act 3 — Join with a code (4 min) ... 8
Act 4 — Honest health (2 min) ... 8
Act 5 — Protocol simulation (optional, 3 min) ... 9
Act 6 — A real WireGuard tunnel (4 min) ... 9
Act 7 — Autopsy: diagnose, approve, verify (4 min) ... 10
Act 8 — AI Insights (2 min) ... 10
Act 9 — Security: revoke and audit (2 min) ... 11
Act 10 — Quality (optional, 1 min) ... 11
5. Questions you may be asked ... 12
6. If something goes wrong, and resetting ... 13
Troubleshooting during the demo ... 13
Resetting for another run ... 13
Documents to have ready ... 13
7. Printable checklist ... 14

Demo at a glance

The story you are telling: NexLink is a private-network platform that only reports what it can prove. Each act shows one idea, in this order:

Act	What you show	Time	Key message
1	Landing page and architecture	3 min	One app, one port; VPN traffic goes device to device over WireGuard
2	Sign up and guided setup	3 min	An account and a private network in a minute; address ranges are validated
3	Join with a code	4 min	Keys are made on the computer; nothing secret is copied; an operator approves
4	Honest health	2 min	“Enrolled” is not “connected” — no evidence means unknown
5	Protocol simulation (optional)	3 min	The packet sequence, slowed down and clearly labelled as a replay
6	A real WireGuard tunnel	4 min	Connected only after a real handshake; direct first, relay when needed
7	Autopsy: diagnose, approve, verify	4 min	Evidence, policy engine, human approval, verified result
8	AI Insights	2 min	AI explains with receipts; it never acts on its own
9	Security: revoke and audit	2 min	A revoked device is locked out at once; everything is audited
10	Quality (optional)	1 min	128 automated tests across Node.js, Go and Python



The architecture you explain in Act 1 (also in the Project Report).

1. Readiness check (24 October 2026)

Everything below was checked on this PC. Use it to answer “does this really work?”

Component	Check	Result
Web app	npm test	85 / 85 passed
Networking agent (Go)	go test ./...	28 / 28 passed
Diagnostic service (Python)	python -m pytest -q	15 / 15 passed
Production build	npm run build (22 Oct)	Passed, 19 routes
Agent binary	networking-agent\bin\nexlink-agent.exe version	0.2.0
Real tunnel, direct	maitrey ⇄ maitrey-peer with run --userspace	Handshake fresh, 0.42–0.48 ms
Real tunnel, relay	Same agents, first start on the relay (22 Oct)	Handshake in 1 s, ~1–3.5 ms
Console verdict	Network page for maitrey-net	Operational, 1/1 verified
Landing and sign-in pages	Rendered in a fresh browser profile	Correct
Docker Desktop	Fails to start on this PC	Not needed for this demo

Be careful

Use a fresh database for the demo. The development database still contains demo and test networks; offline devices and old incidents look like a broken system on screen. Section 2.2 shows the one-line change.

Acts 3 and 6 run two agents on one PC: the real WireGuard protocol, but one machine. If a second computer is available, the optional Act 6c connects it for real.

2. The day before

2.1 Update and install

Open PowerShell in the project folder (D:\projects\cn):

```
git pull
.\scripts\start-all.ps1 -Install     # installs web and the AI service, builds the agent downloads
```

2.2 Use a clean demo database

Open web\.env and point MONGODB_URI at a new database name:

```
MONGODB_URI=mongodb://127.0.0.1:27017/nexlink_demo
```

The new database is created automatically. The first account you sign up becomes the administrator (needed for approvals). Change the name back to nexlink afterwards for your normal data.

2.3 Decide on AI (Acts 7 and 8)

- **With AI:** put your Groq key in web\.env as `GROQ_API_KEY=gsk_…`. AI runs inside NexLink — no second service. The file is git-ignored, so the key never reaches GitHub.
- **Without AI:** leave it out. Briefings show that AI is not configured, deterministic diagnosis still works, and the rest of the demo is unchanged.

2.4 Rehearse once

- Run the whole guide once with the clean database, then reset it (section 6) so the real demo starts empty.
- Set the browser zoom to 80–90% so whole pages fit on the projector.
- Close anything using port 3000 and old agent windows from earlier runs.
- Keep the Project Report (PDF) open in a tab for questions.

3. Starting everything (10 minutes before)

Open two PowerShell windows and give each a clear title.

Window 1 — NexLink (console, API, live updates and relay on one port)

```
cd D:\projects\cn\web
npm run dev
# wait for: NexLink ready on http://localhost:3000
```

Window 2 — Agents

```
cd D:\projects\cn\networking-agent\bin
.\nexlink-agent.exe version     # prints 0.2.0
```

Quick checks

- http://localhost:3000/healthz shows `{"ok":true,"service":"nexlink"}`.
- http://localhost:3000 shows the landing page with “systems online” on the sign-in page’s status bar.
- If a page shows “control plane unreachable”, Window 1 is still starting or was stopped — wait and refresh.

4. The demo, act by act

Act 1 — Landing page and architecture (3 min)

1. Open http://localhost:3000 — the landing page: “your devices. your network. no limits.”
2. Point at the tunnel readout and the strip at the bottom: real measured values, the 180-second health rule, WireGuard’s ChaCha20-Poly1305.
3. Scroll to **How it works** and **Security**, then show the architecture figure from this guide.



The landing page (24 Oct 2026).

What to say

“NexLink lets you create private networks between your own computers, see their real connectivity, diagnose problems and run approved actions.”

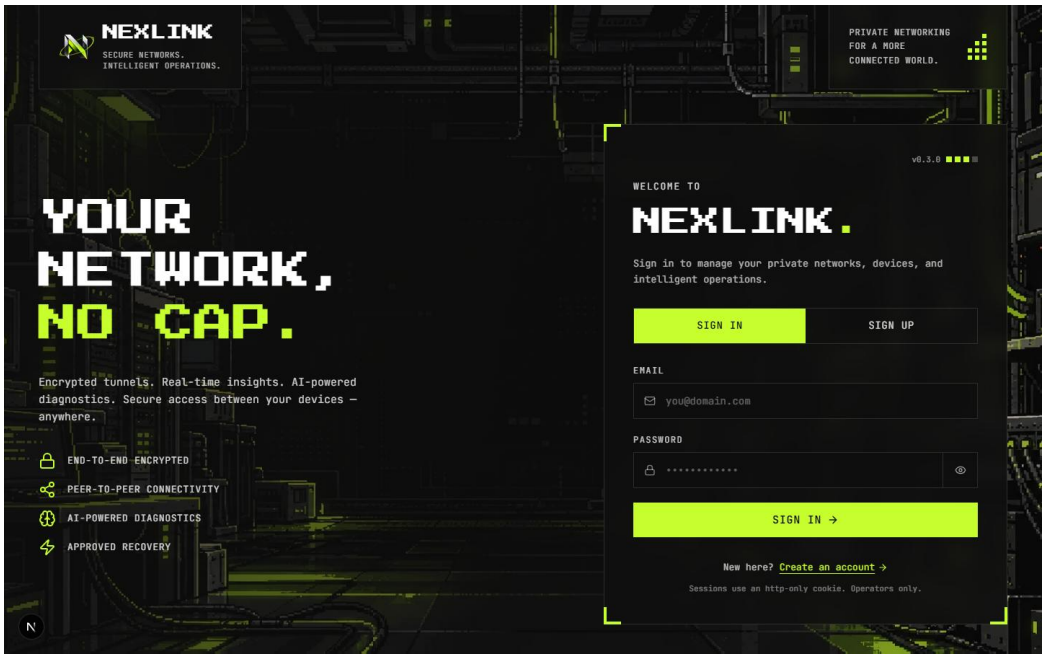
“The whole platform is one app on one port — console, API, live updates and a relay — so it deploys as one container.”

“The server only coordinates. VPN traffic goes directly between devices over WireGuard; when a firewall blocks that, it goes through a relay that only ever sees encrypted packets.”

“The design rule: never claim something we cannot prove. A device is connected only after WireGuard reports a real handshake.”

Act 2 — Sign up and guided setup (3 min)

1. Click **[get started →]**. The sign-in page opens on the **sign up** tab.
2. Enter your name, email and a password (12+ characters) → **create account**. You land on **setup**.
3. Create the network: name **maitrey-net**. Point out the suggested free private range — the CIDR is optional.
4. Optional: on **networks** → **new network**, try 8.8.8.0/24 (not private) or an overlapping range and show the error.



The sign-in page.

> **What to say**
>
> “The first account becomes the administrator; later accounts only see their own networks.”
>
> “Address allocation is enforced by a unique index in the database, so two devices can never get the same address — even if they join at the same moment.”

Act 3 — Join with a code (4 min)

In Window 2, start the first agent’s join (on another computer this is the one-line `irm … | iex` from **add a device**):

```
.\nexlink-agent.exe join --server http://127.0.0.1:3000 --state-dir .\stage\maitrey
```

1. The window prints a short code such as **KXRT-4P2M**, the computer name “Maitrey” and a key fingerprint.
2. In the browser: **add a device** → enter the code → check the computer name → name **maitrey** → network **maitrey-net** → **approve & enroll**.
3. The agent window prints “approved: enrolled as maitrey … address 10.x.x.2”.
4. Repeat with a second folder and name it **maitrey-peer**:

```
.\nexlink-agent.exe join --server http://127.0.0.1:3000 --state-dir .\stage\maitrey-peer
```

> **What to say**
>
> “The agent made its own keys on the PC — an Ed25519 identity key and a WireGuard key. The private keys never leave the device.”
>
> “The line I ran is the same for every computer and contains nothing secret. The code proves which computer is asking, and an operator decides whether it gets in.”
>
> “From now on every request the agent makes is signed, time-stamped and replay-protected.”

Act 4 — Honest health (2 min)

1. **devices**: maitrey and maitrey-peer show enrolled, with no heartbeat yet.
2. **topology**: the two devices are joined by a dotted “configured” line, not a lime one.

3. Open **maitrey**: identity, connection and performance are separate sections; with no evidence they say unknown.

What to say

“They are enrolled, but nothing has proven they can talk, so NexLink says unknown — not healthy, not zero.”

“A heartbeat proves only that the agent reaches the server. It says nothing about the tunnel.”

Act 5 — Protocol simulation (optional, 3 min)

1. Sidebar → **simulation**, scenario normal tunnel. Point to the “simulated packets” label.
2. Narrate the phases: enrollment → peer config → handshake → encrypted traffic → telemetry. Pause on the handshake and open the packet inspector (Handshake Initiation, 148 bytes).
3. Switch to **control plane down**: the server goes red, but probes keep flowing through the tunnel.

What to say

“This is a replay of our protocol, not live traffic, and every packet is labelled. If the server goes down, the tunnel keeps working: the keys are already on both PCs.”

Act 6 — A real WireGuard tunnel (4 min)

In Window 2 start both agents with the production command (two separate windows or tabs):

```
.\nexlink-agent.exe run --userspace --server http://127.0.0.1:3000 --state-dir .\stage\maitrey --
listen-port 51840
.\nexlink-agent.exe run --userspace --server http://127.0.0.1:3000 --state-dir .\stage\maitrey-peer -
-listen-port 51841
```

Be careful

Windows may ask about network access for nexlink-agent.exe — Allow (traffic stays on this PC).

Start the second agent a few seconds after the first. If both start at the same moment, neither has published its address yet and they connect through the relay instead — still encrypted, and a good moment to explain the relay. Restarting maitrey-peer gives the direct path.

1. Each agent prints “tunnel peers updated … maitrey (direct)” and “tunnel probe … rtt_ms=0.4 handshake=1s ago”.
2. **networks** → **maitrey-net**: within about a minute, **OPERATIONAL — 1/1 peer pairs verified by fresh handshake**; both devices healthy.
3. **topology**: the line turns solid lime, labelled **direct**, with packets moving along it.
4. **devices** → **maitrey**: the observatory and the live agent console stream the agent’s own output.

What the audience sees

For the first minute the console may still say “last handshake: never”. That is honest: the first report was sent before the handshake finished. It changes at the next report.

What to say

“This is the real WireGuard protocol — wireguard-go, the official implementation, running inside the agent. No VPN app and no administrator rights.”

“The line turned lime only because WireGuard reported a handshake, not because a record exists.”

Optional 6b — the relay

Stop maitrey-peer (Ctrl+C) and start it again with `--force-relay`. Topology now labels the link **relay**.

> **What to say**
> “When a firewall or carrier-grade NAT blocks direct UDP, the agents send WireGuard packets through the server. The server forwards ciphertext it cannot read, only between devices of the same network, and cuts it the moment a device is revoked.”

Optional 6c — a second computer

1. On the second computer open the portal by this PC’s address (same Wi-Fi), or a deployed portal from any network.
2. Run the line shown on **add a device**: `irm 'http://<this PC>:3000/join.ps1' | iex`. It shows a code with that computer’s own name.
3. Approve it here. Topology shows the new link as direct or relay within a minute.

Same Wi-Fi only: once, as Administrator on this PC, run `.\scripts\lan-firewall.ps1 -Server -Agent`. Different networks need a deployed portal (docs/deploy.md).

Act 7 — Autopsy: diagnose, approve, verify (4 min)

1. **autopsy** → **test event** → device maitrey, category peer handshake stale, peer maitrey-peer, note “demo drill” → create.
2. Open the new [TEST] incident: investigation states, the path view (control plane → maitrey → maitrey-peer) and the diagnostic checks.
3. **run diagnostics** (and **ask ai** if AI is on): evidence from read-only tools → observations citing evidence → hypotheses marked untested or plausible.
4. Proposed recovery: on run_connectivity_test click **approve & run**.
5. Within 10–20 s: **verified outcome: successful — 3/3 measurements valid**.

> **What to say**
> “Diagnosis only uses read-only tools. Every proposed action goes through a deterministic policy engine and then needs a human to approve it.”
>
> “The agent checks again with its own allowlist, runs the test, and the server verifies the result from fresh measurements. Test incidents are never closed automatically.”

Act 8 — AI Insights (2 min)

1. Sidebar → **ai insights**. Show the report for maitrey-net: summary, numbered findings (F-01 …) with their evidence references, risks with plausible / untested / ruled-out tags, uncertainties.
2. Point to the footer: “grounded · N evidence records · explains, never acts”. Optionally click **regenerate**.
3. Scroll to the analytics: latency, loss, incidents and per-device tunnel quality — computed from agent records.

> **What to say**
> “The AI reads the same evidence you see. Every finding cites a recorded fact; a citation that does not exist is rejected before it reaches this page.”
>
> “The AI never receives private keys, passwords or session tokens, and VPN connectivity never depends on it.”

Act 9 — Security: revoke and audit (2 min)

1. **devices** → **maitrey-peer** → **revoke** → reason “lost laptop”.
2. Its agent window immediately reports DEVICE_REVOKED; the topology link disappears.
3. **audit log**: show device enrollment, approval, action approval and verification, revocation, and a denied agent request.
4. Ctrl+C both agent windows.

> **What to say**
>
> “Revocation takes effect on the device’s very next request — its signature is no longer accepted, and its relay connection is closed.”
>
> “Every security-sensitive event is written by the backend to an append-only audit log; there is no API to edit or delete it.”

Act 10 — Quality (optional, 1 min)

```
cd D:\projects\cn\networking-agent
..\.tools\go\bin\go.exe test ./...      # 28 tests
cd ..\diagnostic-service
.\.venv\Scripts\python.exe -m pytest -q     # 15 tests
```

Do not run the web tests or a production build while Window 1 is serving the demo. The web suite (85 tests) can be shown from the readiness table.

> **What to say**
>
> “There are 128 automated tests across Node.js, Go and Python, including a test that brings up a real WireGuard tunnel and checks encrypted traffic.”

5. Questions you may be asked

Question	Answer
Why doesn’t VPN traffic go through your server?	It would be a bottleneck and a privacy risk. The server distributes keys, addresses and policy; WireGuard connects devices directly. Only when a direct path is impossible does the relay forward packets — and those are encrypted end to end.
Is this real WireGuard or a simulation?	Real. The agent embeds wireguard-go, the official userspace implementation. The simulation page is a separate, clearly labelled replay.
What happens behind a strict NAT?	The agent tries direct UDP first; if no handshake comes within 20 seconds it switches that peer to the relay. The topology shows direct or relay per link.
Once on the relay, does it switch back to direct?	Not while the relay link keeps working — that is a known gap we plan to fix. Restarting an agent after its peer has published its address gives a direct path.
How do you stop a device pretending to be another?	Each device has an Ed25519 key generated on the device; every request is signed over method, path, time, a one-time nonce, device id and body hash.
Why is joining safe if the command has no secret?	The computer signs its code request with its own key, and nothing happens until an operator approves the code after checking the computer name and key fingerprint.
How do you know a device is healthy?	Separate signals: agent heartbeat, interface state, and a fresh handshake with each peer. Missing evidence shows as unknown, never healthy.
What does the AI actually do?	It reads evidence collected by read-only tools and writes briefings and diagnoses. Output must match a schema and cite real evidence; actions must be allowlisted, pass the policy engine and be approved by a human.
What if the AI is wrong or down?	Wrong output is rejected by validation; deterministic rules work without AI; networking never depends on it.
How would you deploy it?	One container and MongoDB: Docker Compose behind Caddy for HTTPS, or a PaaS such as Render or Railway with MongoDB Atlas.
Has it run between two separate computers?	Both agents in the verified runs were on one PC. The second-computer path uses exactly the same agent and join line; it is the next test.
How did you test it?	128 automated tests (85 web, 28 Go, 15 Python) and real tunnel runs on 22 and 24 October over both the direct path and the relay.

6. If something goes wrong, and resetting

Troubleshooting during the demo

What you see	What to do
Sign-in shows “control plane unreachable”	Window 1 is still starting or stopped — wait for “NexLink ready”, then refresh. Never run npm run build while it is serving.
Every page shows HTTP 500; Window 1 prints “Cannot find module ./611.js”	Corrupted dev cache: Ctrl+C, then `Remove-Item -Recurse -Force .next; npm run dev` in web.
The join code expired	Codes last 15 minutes — run the join command again for a new one.
“already enrolled as …”	That --state-dir is already used — pick a new folder.
Console says “last handshake: never” after the agents started	Wait for the next report (up to a minute) and refresh; check the agent window shows a handshake.
Link shows relay although both agents are on this PC	They started at the same moment. Restart maitrey-peer to get the direct path — or use it to explain the relay.
Agent prints “heartbeat failed … deadline exceeded”	Its --server points at an old address; on this PC use http://127.0.0.1:3000.
Approved test shows “timed out”	The agents were not running when you approved — start them (Act 6) and approve a new test.
AI panels say AI is not configured	GROQ_API_KEY is not set in web\.env — skip the AI parts or add the key and restart Window 1.
Second computer cannot open the portal	Same Wi-Fi: check this PC’s address with ipconfig and the firewall script. Different network: the portal must be deployed.

Resetting for another run

1. Ctrl+C in every window.
2. Use a new database name in web\.env (e.g. nexlink_demo2), or switch back to nexlink for your normal data.
3. Delete networking-agent\bin\stage (it contains the demo agents’ private keys; git ignores it anyway).

Documents to have ready

Document	Use it for
docs/NexLink-Project-Report.pdf	Full report: architecture, status, test results
docs/run-commands.md	Starting by hand, adding computers, remote sessions
docs/deploy.md	Putting NexLink on the internet
docs/architecture.md, network-protocols.md	Deeper architecture and protocol questions
docs/threat-model.md	Security questions
docs/two-pc-test.md	Connecting a second physical computer

7. Printable checklist

- ☐ git pull; .\scripts\start-all.ps1 -Install
- ☐ web\.env uses MONGODB_URI=…/nexlink_demo (fresh database)
- ☐ Decided on AI: GROQ_API_KEY in web\.env, or no AI
- ☐ Rehearsed once, then reset the database
- ☐ Old agent windows and anything on port 3000 closed
- ☐ Window 1: NexLink ready on http://localhost:3000; /healthz ok
- ☐ Window 2: PowerShell in networking-agent\bin
- ☐ Browser zoom 80–90%; Project Report open in a tab
- ☐ Act 2: account created, maitrey-net created
- ☐ Act 3: maitrey and maitrey-peer approved
- ☐ Act 6: both agents running; maitrey-net operational; topology direct
- ☐ Act 7: approved test verified successful
- ☐ Act 9: revoke shown, audit log shown, agents stopped